2025 IEEE 19th International Conference on Big Data Science and Engineering (BigDataSE) | 979-8-3315-7366-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/BigDataSE66491.2025.00015

A Heterogeneity-aware Training Framework for Large-scale Geo-distributed Deep Learning

Renchao Xie[1,2,3], Zhe Deng[1,2], Qinqin Tang[1,2], Li Feng[1], Ran Zhang[1], Hongye Zhou[1], and Tao Huang[1,3]
[1]State Key Laboratory of Networking and Switching Technology,
Beijing University of Posts and Telecommunications, Beijing, China
[2]Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China
[3]Purple Mountain Laboratories, Nanjing 211111, China

***Abstract*—As datasets and model complexity increase, Geo-distributed deep learning (GeoDL) trains models across multiple sites to avoid single-site capacity limits. Unlike relatively homogeneous intra-datacenter networks, wide-area networks (WANs) exhibit significant heterogeneity and variability, leading to synchronization and communication bottlenecks when resource-limited nodes become stragglers. To tackle this challenge, we propose a heterogeneity-aware GeoDL framework that employs a hierarchical federated tree synchronization mechanism to coordinate the computational and transmission pacing of heterogeneous nodes, thereby enhancing global training efficiency. Additionally, we develop an adaptive scheduling algorithm based on distributed Bayesian optimization (BO) and validate its effectiveness through emulation-based simulations.**

***Index Terms*—Geo-distributed deep learning, distributed training paradigm, synchronous communication mechanism, potential game, network-topology**

I. Introduction

Recently, distributed training has become an essential approach to meet the massive computational and storage demands of large-model training. By integrating compute resources from geographically dispersed data centers, Geo-distributed deep learning (GeoDL) enables multi-center collaboration without migrating raw data [1], thereby fully exploiting remote data resources and yielding models with stronger generalization [2].

To reduce the communication overhead of distributed training, numerous techniques have been explored—including gradient quantization and compression [3], pipeline parallelism [4], and communication-efficient topologies (e.g., Ring-AllReduce [5] and Megatron-LM 3D parallelism)—to improve synchronization efficiency. However, these techniques have been primarily designed for intra-datacenter environments characterized by high bandwidth and low latency, and their performance gains degrade markedly in adverse wide-area network (WANs) settings [6].

Existing optimization techniques fail to address the fundamental challenge of GeoDL—the nonlinear coupling between communication latency and computational efficiency under highly heterogeneous and highly dynamic network conditions [7]. To address this issue, we propose a heterogeneity-aware GeoDL framework, enabling participating data centers to adaptively adjust the number of local training iterations based on real-time computing and networking resource status. Integrated with an efficient synchronization mechanism and scheduling algorithm, this framework can flexibly coordinate the computation-communication pacing across data centers, significantly reducing synchronization overhead and improving overall resource utilization. Our contributions are as follows.

- We present a heterogeneity-aware GeoDL framework that employs an adaptive hierarchical federated tree (HFT) synchronization mechanism to mitigate the communication bottlenecks of traditional star and ring topologies.
- We formalize the tension between synchronization overhead and consistency requirements within a potential-game framework and develop an improved distributed Bayesian optimization algorithm to compute a Nash equilibrium.
- We conduct emulation-based simulations to evaluate the proposed synchronization mechanism. Experimental results indicate that HFT synchronization mechanism achieves consistent and significant training acceleration compared with mainstream synchronization schemes.

II. System Description

In this section, we present the system architecture of the training framework and its implementation details.

A. Training Framework Design

We design a three-layer GeoDL framework to tackle the inefficiency of data synchronization inherent in the traditional parameter-server architecture under the WAN environment. The overall architecture is shown in Fig. 1.

- **Data plane:** The global parameter server serves as a central hub for all parameter updates during every synchronization period. Geographically dispersed data centers perform on-device training on their local datasets and periodically synchronize their model data.
- **Control plane:** A central controller handles global scheduling and process management, coordinating nodes to complete cross-domain collaborative training.
- **Operations plane:** The monitoring component is responsible for tracking computational and network resource utilization to facilitate efficient training progress.

Corresponding author: Qinqin Tang. Email: qqtang@bupt.edu.cn

979-8-3315-7366-9/25/$31.00 ©2025 IEEE
DOI 10.1109/BigDataSE66491.2025.00015

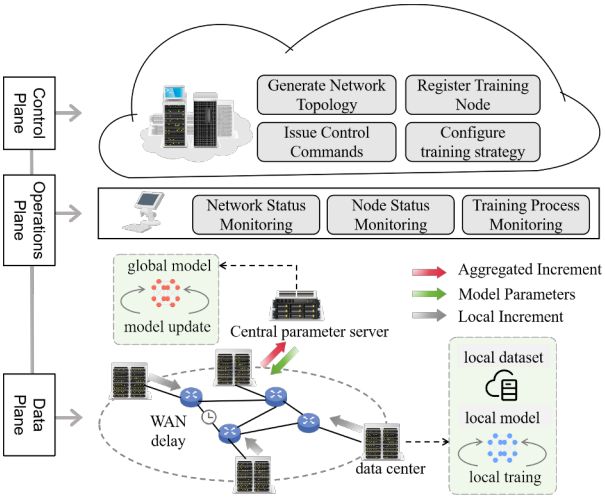


Fig. 1: Framework of the heterogeneity-aware GeoDL architecture.

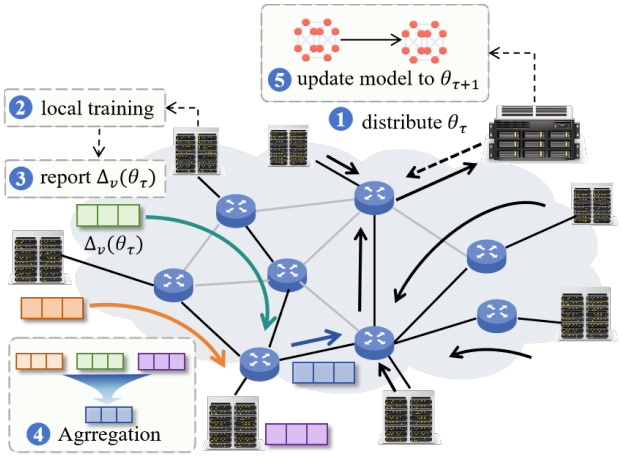


Fig. 2: Workflow of HFT synchronization mechanism.

Fig.1 illustrates a GeoDL system, comprising a set of data centers with heterogeneous computing capabilities, which is denoted as $\mathbb{V} = [v_0, v_1, \ldots, v_{|\mathbb{V}|}]$ and referred to as training nodes in the following. A central controller coordinates the process based on real-time computing and network status. Training proceeds in synchronization cycles where each node: (1) completes local training, (2) performs in-situ gradient aggregation via hierarchical synchronization mechanism during transmission, and (3) receives updated parameters to initiate the next round, repeating until convergence. The synchronization mechanism is detailed next.

B. Hierarchical Synchronous Transmission Mechanism

This framework utilizes a hierarchical federated tree (HFT) synchronization mechanism based on a tree topology for sequential gradient aggregation [8], [9]. Data originates from leaf nodes and is forwarded upstream through the tree. The workflow of the HFT synchronization mechanism is shown in Fig. 2.

Each synchronization cycle begins with all nodes receiving the latest global model parameters. Subsequently, each node trains locally and transmits its gradient to the designated parent node. Internal nodes aggregate gradients from their child nodes and their own, then transmit the merged results upstream. Finally, the root node pushes the aggregated gradient to the global parameter server, completing the synchronization cycle. This hierarchical approach eliminates raw-data transmission and significantly reduces communication overhead.

C. Distributed Training Paradigm

Set the global objective function as:

$$F(\theta) = \frac{1}{|\mathbb{V}|} \sum_{v \in \mathbb{V}} \mathbb{E}_{s_v \sim S_v} p_v f_v(\theta; s_v), \tag{1}$$

where S_v is the entire local training dataset of node v and s_v is mini-batch sampled from S_v with fixed size. $f_v(\cdot)$ is the loss function of node v. p_v is the data weight of node v, which is defined as:

$$p_v = \frac{|S_v|}{\sum_{v \in \mathbb{V}} |S_v|}. \tag{2}$$

To reduce the data volume transmitted over the backbone network, the training nodes adopt a low-frequency synchronization method. Let θ_τ be the current global parameter distributed by the server at the beginning of the τ-th synchronization period. Each node performs stochastic gradient descent using mini-batches sampled from its local dataset with a learning rate of η. Here, $\theta^v_{\tau,h}$ denotes the updated model parameter of node v after h local training iterations based on θ_τ. Set the total number of local training iterations in each synchronization period is H_v , then local model parameter in node v is updated to:

$$\theta^v_{\tau,H_v} \leftarrow \theta_\tau - \eta \sum_{h=1}^{H_v} \nabla f_v\left(\theta^v_{\tau,h-1}; s_v\right). \tag{3}$$

Using parameter differences to approximate the effective gradient accumulated over H_v local training iterations. The local model increment of node v in the τ-th synchronization period can be expressed as:

$$\Delta_v(\theta_\tau) = \theta_\tau - \theta^v_{\tau,H_v}. \tag{4}$$

When an internal node v performs aggregation, it assigns different weights to the increments from itself and from its child nodes.

$$\omega_{v,n} = \begin{cases} \dfrac{|S_v|}{\sum\limits_{i \in \mathbb{V}_{\text{sub},v}} |S_i|}, & \text{if } n = v \\ \dfrac{\sum\limits_{i \in \mathbb{V}_{\text{sub},n}} |S_i|}{\sum\limits_{i \in \mathbb{V}_{\text{sub},v}} |S_i|}, & \forall n \in \mathbb{V}_{\text{chi},v} \end{cases}, \tag{5}$$

where $\mathbb{V}_{\text{chi},v}$ denotes the set of child nodes of node v, $\mathbb{V}_{\text{sub},v}$ denotes the set of nodes contained in the subtree that takes

node v as the root. $|S_i|$ denotes the total sample size of node i. Then the aggregated increment computed by node v is:

$$\begin{aligned}\Delta_{\text{agg},v}(\theta_\tau) &= \sum_{n\in\mathbb{V}_{\text{chi},v}} \omega_{v,n}\Delta_{\text{rep},n}(\theta_\tau) + \omega_{v,v}\Delta_v(\theta_\tau) \\ &= \frac{\sum\limits_{i\in\mathbb{V}_{\text{sub},v}} |S_i|\Delta_i(\theta_\tau)}{\sum\limits_{i\in\mathbb{V}_{\text{sub},v}} |S_i|}\end{aligned}, \tag{6}$$

where $\Delta_{\text{rep},n}(\theta_t)$ is the increment reported by child node n. If $\mathbb{V}_{\text{chi},n} \neq \emptyset$, $\Delta_{\text{rep},n}(\theta_\tau) = \Delta_n(\theta_\tau)$; otherwise, $\Delta_{\text{rep},n}(\theta_\tau) = \Delta_{\text{agg},n}(\theta_\tau)$. Similarly, the final aggregated increment $\Delta_{\text{agg},v_r}(\theta_\tau)$ submitted by the root node v_r to the global parameter server is:

$$\Delta_{\text{agg},v_r}(\theta_\tau) = \frac{\sum\limits_{i\in\mathbb{V}} |S_i|\Delta_i(\theta_\tau)}{\sum\limits_{i\in\mathbb{V}} |S_i|}. \tag{7}$$

Then global parameter server updates the global model as:

$$\theta_{\tau+1} \leftarrow \theta_\tau - \Delta_{\text{agg},v_r}(\theta_\tau). \tag{8}$$

D. Synchronization Latency Model

The synchronization latency of a federated tree includes not only the data transmission time but also the waiting time induced by lagging child nodes and local communication blocking, as aggregating nodes must complete local training and receive all increments from child nodes before performing aggregation.

In a given tree topology T_{v_r} rooted at v_r, assume the local training time is proportional to the product of per-iteration computation time and the number of local training iterations. Taking the training start time as the synchronization reference, the moment when node v submits increments to its parent is given by:

$$\text{send\_t}_v = \begin{cases} t_v h_v, & \text{if } \mathbb{V}_{\text{chi},v} = \emptyset \\ \max\left\{T_{\text{recv}},\ t_v h_v\right\}, & \text{otherwise} \end{cases}, \tag{9}$$

where $T_{\text{recv}} = \max\limits_{n\in\mathbb{V}_{\text{chi},v}} \{\text{send\_t}_n + d(e_{n\to v})\}$, $d(e_{n\to v})$ represents the transmission latency from node n to node v. Define the longest idle time within a single period among all nodes as synchronization delay. Further the synchronization latency of federated tree T_{v_r} can be expressed as:

$$L_{T_{v_r}} = \text{send\_t}_{v_r} + d_{\text{trans}}(v_r) - \min_{v\in\mathbb{V}} \{\text{send\_t}_v\}, \tag{10}$$

where $d_{\text{trans}}(v_r)$ is the transmission latency from v_r to the global parameter server.

E. Potential Games of Distributed Training

In the HFT synchronization mechanism, all training nodes pursue the same global objective: to minimize the weighted loss of all nodes in the shortest time. However, because nodes differ in compute speed and network throughput, stragglers force faster nodes to wait, stalling synchronization. To address this issue, an intuitive approach is to encourage high-performance nodes to perform additional local iterations while waiting for stragglers. This reduces idle waiting time and allows these nodes to explore higher-quality local models [10]. However, if nodes overly prioritize increasing the number of local training iterations, it will reduce the global synchronization frequency, leading to parameter drift [11]. Therefore, the training strategies between nodes align with a quasi-cooperative game.

Here we introduce a game theory framework $\mathcal{G} = \{\mathbb{V}, \mathbb{H}, [u_v]_{v\in\mathbb{V}}\}$, where $\mathbb{H}$ denotes the global joint strategy space, and u_v represents the utility function of node v. A joint strategy in $\mathbb{H}$ can be expressed as $H = (h_v, h_{-v})$, where h_v represents the number of local training iterations of node v, and $h_{-v} = [h_{v'}]_{v'\in\mathbb{V}\setminus\{v\}}$ represents the external strategies from the perspective of node v.

The design of the node utility function is based on a quasi-cooperative relationship between nodes. The utility function of node v is defined as:

$$\begin{aligned}u_v(H) = -\lambda_1 \cdot \min_{\tau\in E_T} V(\theta_\tau|H) - \lambda_2 \cdot L(h_v, h_{-v}) \\ +\lambda_3 \cdot h_v - \lambda_4 \cdot L(h_{-v}),\end{aligned} \tag{11}$$

where E_T is the set of synchronization cycles within the duration T, $\min_{\tau\in E_T} V(\theta_\tau|H)$ denotes the minimum validation loss achieved during T. And $L(h_v, h_{-v})$ represents the global synchronization latency under joint strategy H, $L(h_{-v})$ represents the global synchronization latency when node v does not participate in training but only acts as a relay node for necessary data forwarding. Note that $L(h_v, h_{-v}) \geq L(h_{-v})$, and a larger difference between these two values implies a stronger impact of node v on the global synchronization latency and a shorter idle waiting time in a synchronization period. The coefficients $\lambda_1, \lambda_2, \lambda_3, \lambda_4 > 0$ are weight factors used to balance dimensions.

Define the potential function based on global object as:

$$\varphi(H) = -\lambda_1 \cdot \min_{\tau\in E_T} V(\theta_\tau|H) - \lambda_2 \cdot L(h_v, h_{-v}) - \lambda_3 \cdot \sum_{v\in\mathbb{V}} h_v. \tag{12}$$

When node v changes its strategy from h_v to h'_v. Define $H = (h_v, h_{-v})$ and $H' = (h'_v, h_{-v})$. The resulting change in its utility can be denoted as:

$$\begin{aligned}&u_v(H') - u_v(H) \\ &= \left[-\lambda_1 \min_{\tau\in E_T} V(\theta_\tau|H') - \lambda_2 L(H') + \lambda_3 h'_v - \lambda_4 L(h_{-v})\right] \\ &\quad - \left[-\lambda_1 \min_{\tau\in E_T} V(\theta_\tau|H) - \lambda_2 L(H) + \lambda_3 h_v - \lambda_4 L(h_{-v})\right] \\ &= -\lambda_1 \left[\min_{\tau\in E_T} V(\theta_\tau|H') - \min_{\tau\in E_T} V(\theta_\tau|H)\right] \\ &\quad - \lambda_2 \left[L(H') - L(H)\right] \\ &\quad + \lambda_3 \left[\sum_{i\in\mathbb{V}\setminus\{v\}} h_i + h'_v - \sum_{i\in\mathbb{V}\setminus\{v\}} h_i - h_v\right] \\ &= \varphi(H') - \varphi(H)\end{aligned} \tag{13}$$

Since every node's unilateral deviation produces a payoff change equal to the change in the global potential function,

the game is an exact potential game. Moreover, because the strategy space is finite, standard results for finite potential games ensure the existence of at least one Nash equilibrium.

III. Distributed Solution For Joint Optimization Problem

In this section, we decouple the training strategy from the optimization problem of aggregation topology and propose an improved distributed Bayesian optimization to jointly solve the Nash equilibrium solution.

A. Fastest Topology Construction Algorithm

Under a given training strategy, Algorithm 1 shows how to construct a set of candidate topologies with the smaller synchronization latency. First, using Dijkstra's algorithm for each node pair (i, j) in graph G to find the shortest path from i to j, which is returned as a node sequence $p_{i\to j}$. If i is unreachable to j, the node sequence $p_{i\to j}$ returns an empty set. Record $p_{i\to j}$ in set P_i. By merging duplicate path prefixes from the same source, we can obtain a complete set of candidate federated tree topologies.

Algorithm 1 Candidate Topologies Construction Algorithm

Require: Undirected graph $G = (\mathbb{V}, \mathbb{E})$, Link transmission latency matrix $[d(e)]_{e\in\mathbb{E}}$
Ensure: Candidate federated tree set $\mathcal{T} = [T_i]_{i\in\mathbb{V}}$
1: **for** $i \in \mathbb{V}$ **do**
2: $P_i \leftarrow \emptyset$
3: **for** $j \in \mathbb{V} \setminus \{i\}$ **do**
4: Calculate node sequence $p_{i\to j}$ of shortest path from i to j using Dijkstra's algorithm
5: $P_i \leftarrow P_i \cup \{p_{i\to j}\}$
6: **end for**
7: Merge duplicate sequences in P_i to generate aggregation tree topology T_i
8: **end for**
9: **return** $\mathcal{T} = [T_i]_{i\in\mathbb{V}}$

With the root node fixed, synchronization latency is determined solely by the data convergence time from the straggler to the root node, irrespective of preceding nodes' topology. Therefore, ensuring the shortest path from the straggler to the root node is sufficient. Although multiple optimal topologies exist, Algorithm 1 is guaranteed to find one of them.

The controller first calculates the local computation time for each node, using the current training strategy and historical single-iteration runtime data. It then iterates through all the topologies generated by Algorithm 1, calculating the synchronization latency for each. The tree topology with the minimal synchronization latency is selected as the aggregation topology, as shown in Algorithm 2.

B. Improved Distributed Bayesian Optimization Algorithm

To tackle the large solution space of centralized algorithms, this algorithm adopts a distributed architecture where each node run the Bayesian optimization (BO) algorithm in parallel.

Algorithm 2 Optimal Federated Tree Selection Algorithm

Require: Candidate Federated Tree Set $\mathcal{T}$, Number of Local Iterations $[h_v]_{v\in\mathbb{V}}$
Ensure: Root node v_r, Federated tree topology T_{v_r}
1: Compute local completion time: $T_{\text{loc}} \leftarrow [h_v \times t_v]_{v\in\mathbb{V}}$
2: **for** $i \in \mathbb{V}$ **do**
3: Calculate synchronization latency L_{T_i} for tree topology $T_i \in \mathcal{T}$ via (10)
4: **end for**
5: Select topology with minimal latency: $v_r \leftarrow \arg\min_{i\in\mathbb{V}} L_{T_i}$
6: **return** v_r, T_{v_r}

Due to the discreteness of the strategy space, we select random forest regression $\hat{f}_v(h_v, h_{-v})$ as a surrogate model for the utility function $u_v(h_v, h_{-v})$. Each node independently maintains a local surrogate model and search for its optimal response via Expected Improvement (EI) sampling.

- **Proxy model training:** In each round of interaction, the central controller accepts sampled training strategies from distributed nodes, forms a joint strategy to conduct simulation. After a specific training duration T, controller feeds back the joint strategy and corresponding utility to all nodes. Each node adds the joint strategy-utility pair $\left((h_v^{(i)}, h_{-v}^{(i)}), u_v^{(i)}\right)$ to its local observation space D_v.

$$D_v \leftarrow D_v \cup \left((h_v^{(i)}, h_{-v}^{(i)}), u_v^{(i)}\right), \tag{14}$$

  where $u_v^{(i)}$ represents the actual utility value of node v under joint strategy $(h_v^{(i)}, h_{-v}^{(i)})$ in the i-th round. After each iteration, the node uses D_v to train the surrogate model $\hat{f}_v\left((h_v^{(i)}, h_{-v}^{(i)})\right)$.
- **Local Optimal Response Search:** When node v performs strategy sampling in the i-th round, it first applies random perturbation to the current strategy $h_v^{(i)}$ to generate a neighborhood strategy set $C_{v,i+1}$. Each candidate strategy $c_{v,i+1} \in C_{v,i+1}$ is combined with the current strategies of other nodes $h_{-v}^{(i)}$ to form a new candidate joint strategy $a_{v,i+1} = (c_{v,i+1}, h_{-v}^{(i)})$, which is then input to the local surrogate model for value prediction, obtaining the predicted mean $\mu(a_{v,i+1})$ and standard deviation $\sigma(a_{v,i+1})$:

$$\mu(a_{v,i+1}) = \frac{1}{M}\sum_{m=1}^{M} g^{(m)}(a_{v,i+1}), \tag{15}$$

$$\sigma(a_{v,i+1}) = \sqrt{\frac{1}{M}\sum_{m=1}^{M}\left(g^{(m)}(a_{v,i+1}) - \mu(a_{v,i+1})\right)^2}, \tag{16}$$

  where $g^{(m)}$ represents the predicted value of the m-th decision tree for the candidate joint strategy $a_{v,i+1}$, and M represents the number of decision trees in the random forest.

For each candidate joint strategy $a_{v,i+1}$, calculate EI:

$$\mathrm{EI}(a_{v,i+1}) = \left(\mu(a_{v,i+1}) - \max_{i=1,\ldots,N}\{u_v^{(i)}\} - \xi\right), \quad (17)$$
$$\cdot\Phi(z) + \sigma(a_{v,i+1}) \cdot \phi(z)$$

$$z = \frac{\mu(a_{v,i+1}) - \max_{i=1,\ldots,N}\{u_v^{(i)}\} - \xi}{\sigma(a_{v,i+1})}, \quad (18)$$

where ξ is a coefficient for the exploration-exploitation trade-off. During each iteration, the node selects the candidate strategy with the maximum EI as next sampling point, and reports the corresponding $c_{v,i+1}$ as the local strategy $h_v^{(i+1)}$ to controller. Repeat the above process until the joint strategy converges or the algorithm reaches the maximum number of iterations.

To reduce the high computational cost of simulation and sensitivity to initialization, we employ discrete Latin hypercube sampling (LHS) and augment it with high-quality historical samples. This forms a warm-start set for Bayesian optimization, which reduces blind exploration in its early stages. The complete procedure is detailed in Algorithm 3.

IV. Simulation Results And Discussions

In this section, we describe the deployment of a distributed training platform across three physical hosts. The configuration details are shown in Table 1.

TABLE I: Hardware Configuration of Experimental Platform

Device Model	Processor
ROG Strix G512LV	Intel (R) Core (TM) i7-10875H NVIDIA TU106M (GPU)
LAPTOP-7GI3C08C	13th Gen Intel (R) Core (TM) i5-13420H
Dell R740xd Server	Intel (R) Xeon (R) Gold 5218R NVIDIA L40 (GPU)

We simulate a distributed training environment with 12 worker nodes, a global parameter server, and a central controller. The HFT architecture is utilized in this setup and the training strategy is generated by the scheduling algorithm described in Section III. We design multiple image-recognition training tasks with different complexities. To emulate WAN conditions, we configure inter-node network bandwidth between 10 Mbps and 100 Mbps and control link latency between 10 ms and 50 ms.

We choose three other synchronization mechanisms for comparison: Parameter Server (PS), Balanced Binary Tree (BBT), and Fastest Aggregation Tree (FAT). In PS mechanism, all nodes communicate directly with a central server. Both BBT and FAT mechanism use tree topologies for aggregation. But in BBT mechanism, each non-leaf node has at most two child nodes, and the depth difference between all leaf nodes does not exceed 1. FAT mechanism builds a topology via a shortest-path algorithm to minimize transmission latency, but it does not account for the inconsistency of local training times across nodes. In the comparison experiments, each node traverses its local dataset once per synchronization period.

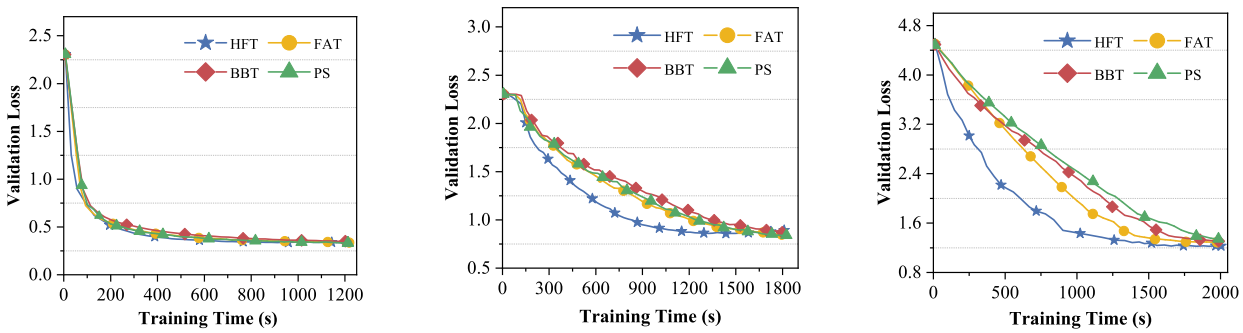
Algorithm 3 Distributed BO for Joint Strategy Solution

Require: Warm Start Set $\mathbb{W}$, Max iterations $I_{\max}$, Convergence threshold δ, Local computing time matrix $T = [t_v]_{v\in\mathbb{V}}$
Ensure: Optimal joint strategy H^*

for each $H \in \mathbb{W}$ **do**
  Controller constructs topology via Alg. 1 & 2
  Run simulation with strategy H
  Controller calculates $U(H) = [u_v(H)]_{v\in\mathbb{V}}$ via (17)
  Distribute $u_v(H)$ to corresponding node
end for
for each $v \in \mathbb{V}$ in parallel **do**
  Initialize D_v and $\hat{f}_v$
  Set $h_v^{(0)} \leftarrow \arg\max_{h_v} \hat{f}_v(h_v, h_{-v})$
end for
for $i = 1$ to $I_{\max}$ **do**
  Controller collect current strategies from all nodes to form joint strategy $H^{(i)} = (h_1^{(i)}, \ldots, h_{|\mathbb{V}|}^{(i)})$
  Run simulation under $H^{(i)}$
  Calculate $U(H^{(i)}) = [u_v(H^{(i)})]_{v\in\mathbb{V}}$
  Distribute $u_v(H^{(i)})$ along with $H^{(i)}$ to each node v
  for each $v \in \mathbb{V}$ in parallel **do**
    $D_v \leftarrow D_v \cup \left((h_v^{(i)}, h_{-v}^{(i)}), u_v(H^{(i)})\right)$
    Update $\hat{f}_v$ on D_v
    Generate $C_{v,i+1}$
    Set $c_{v,i+1}^* \leftarrow \arg\max_{c\in C_{v,i+1}} \mathrm{EI}\left((c, h_{-v}^{(i)})\right)$
    Set $h_v^{(i+1)} \leftarrow c_v^*$
  end for
  if $\|H^{(i)} - H^{(i-1)}\| \le \delta$ or $i = I_{\max}$ **then**
    break
  end if
end for
$H^* \leftarrow H^{(i)}$
return H^*

In the subsequent experiments, each dataset is partitioned across the 12 nodes in a random and unequal manner. All training nodes employ a batch size of 64 and utilize the SGD optimizer with an initial learning rate of 0.05. The weight factors in the utility function are set as $\lambda_1 = 1$, $\lambda_2 = 0.5$, $\lambda_3 = \lambda_4 = 0.25$. The convergence threshold for distributed Bayesian optimization is 10^{-6}, and the surrogate model incorporates 100 decision trees. The duration for a single simulation training run is 15 minutes.

Fig. 3 demonstrates the training performances of several synchronization mechanisms across tasks of different complexity. The specific experimental settings are as follows: Fig. 3(a) presents results for a custom 4-layer CNN on Fashion-MNIST dataset, Fig. 3(b) for Lenet-5 on CIFAR-10 dataset, and Fig. 3(c) for Resnet-18 on CIFAR-100 dataset. Results show that, for lower-complexity tasks, the convergence rates of different synchronization mechanisms are similar. This is because the acceleration advantage of



(a) 4-layer CNN on Fashion-MNIST dataset (b) Lenet-5 on CIFAR-10 dataset (c) Resnet-18 on CIFAR-100 dataset

Fig. 3: Training performance of different synchronization mechanisms on tasks with different complexity.

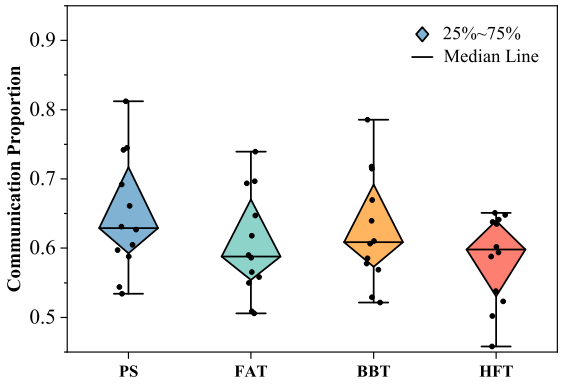


Fig. 4: Communication proportion distribution under different synchronization mechanisms.

high-performance hardware is limited for small models, resulting in weak inter-node heterogeneity. As task complexity increases, lower-performance nodes become stragglers. The HFT mechanism exhibits a clear performance advantage under stronger heterogeneity. On CIFAR-100 dataset, HFT mechanism improves training efficiency by 25.5% over the PS mechanism and also significantly outperforms both the BBT and FAT mechanisms.

Fig. 4 shows the distribution of per-node communication proportions within a synchronization period when training LeNet-5 on CIFAR-10 dataset under different synchronization mechanisms. Leveraging heterogeneous resource awareness, HFT mechanism achieves both a lower mean and reduced variance in communication proportion, indicating better computational resource utilization and improved system load balance.

V. Conclusion

This work presents a heterogeneity-aware GeoDL framework. Theoretically, we formalize the training strategy optimization problem as a potential game among training nodes and develop a distributed BO algorithm for efficient solution. Extensive simulation experiments validate the significant training acceleration achieved by our proposed mechanism.

Acknowledgment

This research was supported in part by the National Natural Science Foundation of China under Grant No. 92367104, in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant No. GML-KF-24-25.

References

[1] L. Liu, Q. Jin, D. Wang, *et al.*, "Psnet: Reconfigurable network topology design for accelerating parameter server architecture based distributed machine learning," *Future Generation Computer Systems*, vol. 106, pp. 320–332, 2020.

[2] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "Adaptive federated learning in resource constrained edge computing systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1205–1221, 2019.

[3] J. Guo, W. Liu, W. Wang, J. Han, R. Li, Y. Lu, and S. Hu, "Accelerating distributed deep learning by adaptive gradient quantization," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1603–1607, 2020.

[4] N. Takisawa, S. Yazaki, and H. Ishihata, "Distributed deep learning of resnet50 and vgg16 with pipeline parallelism," in *2020 Eighth International Symposium on Computing and Networking Workshops*, pp. 130–136, 2020.

[5] Z. Chen, X. Liu, M. Li, Y. Hu, H. Mei, H. Xing, H. Wang, W. Shi, S. Liu, and Y. Xu, "Rina: Enhancing ring-allreduce with in-network aggregation in distributed model training," in *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*, pp. 1–12, 2024.

[6] Z. A. El Houda, D. Nabousli, and G. Kaddoum, "Cost-efficient federated reinforcement learning- based network routing for wireless networks," in *2022 IEEE Future Networks World Forum (FNWF)*, pp. 243–248, 2022.

[7] S. Mukherjee, D. Lu, B. Raghavan, P. Breitkopf, S. Dutta, M. Xiao, and W. Zhang, "Accelerating large-scale topology optimization: State-of-the-art and challenges," *Archives of Computational Methods in Engineering*, vol. 28, pp. 4549 – 4571, 2021.

[8] E. Wu, H. Cui, Z. Chen, and R. E. Welsch, "Treeago: Tree-structure aggregation and optimization for graph neural network," *Neurocomputing*, 2022.

[9] L. Luo, S. Yang, W. Feng, H. Yu, G. Sun, and B. Lei, "Optimizing communication topology for collaborative learning across datacenters," in *International Conference on Emerging Networking Architecture and Technologies*, 2023.

[10] H. Mi, K. Xu, D. Feng, H. Wang, and X. Lan, "Collaborative deep learning across multiple data centers," *Sciece China. Information Sciences*, vol. 63, no. 8, 2020.

[11] Z. Xia, B. Zhong, T. Zhao, K. Li, and S. Zhang, "Federated learning system eliminating model drift in distributed edge computing: Theoretical analytics and application on pit engineering state monitoring," *Adv. Eng. Inform.*, vol. 66, Aug. 2025.